

Actividad en Clase - Investigación: Arquitectura Asíncrona con RabbitMQ y Aplicación al Proyecto Ombú

Universidad de la Cuenca del Plata (UCP)

Facultad de Arte, Diseño y Comunicación / FAITA

Carrera: Ingeniería en Sistemas de Información

Asignatura: Paradigmas y Lenguajes de Programación III

Docente: Prof. Carlos Emiliano Pereyra

Equipo: Grupo 1

Integrantes:

- Lucrecia Sabrina Mencia
- Luciano Rubén Chesani

Proyecto de referencia: Ombú - Cervecería de Barrio (Tap Wall Autoservicio NFC)

Fecha de entrega: 23 de septiembre de 2026 (23:59 hs)

A. ¿Qué es RabbitMQ y cuál es su modelo conceptual?

RabbitMQ es un intermediario de mensajería (*Message Broker*) de código abierto ampliamente adoptado en la industria, desarrollado originalmente en Erlang y basado en el protocolo estándar abierto **AMQP 0-9-1 (Advanced Message Queuing Protocol)**. Su propósito fundamental es actuar como un gestor de colas inteligente y confiable que recibe, enruta, almacena temporalmente y entrega mensajes entre diferentes aplicaciones o componentes de software desacoplados.

A diferencia de los protocolos tradicionales de mensajería directa punto a punto (como sockets TCP o llamadas síncronas HTTP), el modelo conceptual de RabbitMQ introduce una capa de mediación que separa de forma estricta la producción del mensaje de su destino final.



Componentes Clave y su Interacción Técnica

1. **Productor (Producer / Publisher):** Aplicación o servicio cliente que genera información (eventos de negocio, órdenes, notificaciones) y la emite al broker. En la especificación AMQP, **el productor nunca envía mensajes directamente a una cola**, sino que siempre los publica dirigidos a un *Exchange*.
2. **Intercambiador (Exchange):** Componente neurálgico del broker encargado de inspeccionar las cabeceras y la clave de enrutamiento (*Routing Key*) del mensaje entrante para determinar a qué cola o conjunto de colas debe ser remitido. Existen cuatro tipologías estándar:
 - **Direct:** Enruta el mensaje a aquellas colas cuya clave de enlace (*Binding Key*) coincida exactamente con la *Routing Key*.
 - **Fanout:** Difusión masiva (*broadcast*). Ignora la *Routing Key* y replica el mensaje a todas las colas vinculadas al exchange.
 - **Topic:** Enrutamiento semántico avanzado por patrones jerárquicos de palabras separadas por puntos (utilizando comodines `*` para exactamente una palabra y `#` para cero o más palabras).
 - **Headers:** Enrutamiento basado en atributos y metadatos de cabecera en lugar de la *Routing Key*.
3. **Cola (Queue):** Estructura secuencial de almacenamiento (buffer de memoria o disco) que conserva los mensajes bajo el principio FIFO (*First In, First Out*) hasta que un consumidor

los solicita o los procesa. Poseen atributos clave de resiliencia: durabilidad (`durable: true`), exclusividad y auto-eliminación.

4. **Enlace (*Binding*):** Regla o configuración que asocia formalmente un *Exchange* con una *Queue*, definiendo qué mensajes debe capturar dicha cola.
5. **Clave de Enrutamiento (*Routing Key*):** Metadato contextual provisto por el productor al publicar el mensaje, empleado por el algoritmo del *Exchange* para evaluar los *Bindings*.
6. **Consumidor (*Consumer / Worker*):** Servicio receptor suscrito a una cola que extrae y procesa los mensajes disponibles.
7. **Conexión y Canales (*Connection & Channels*):** Una *Connection* representa una conexión TCP física entre la aplicación y el broker. Sobre esta única conexión física se abren múltiples *Channels* (canales virtuales multiplexados y livianos), optimizando drásticamente el uso de recursos de red y evitando la sobrecarga de negociar sockets en cada operación.

B. ¿Para qué sirve y qué problemas de arquitectura resuelve?

En arquitecturas distribuidas modernas, la comunicación exclusivamente síncrona mediante APIs REST genera un acoplamiento estrecho que vulnera la estabilidad del sistema ante escenarios de alta concurrencia o indisponibilidad parcial. RabbitMQ resuelve patologías arquitectónicas estructurales:

1. Desacoplamiento Espacial y Temporal

- **Espacial:** Los servicios productores desconocen la dirección IP, puerto, lenguaje o topología de los consumidores. Solo conocen el nombre del *Exchange*.
- **Temporal:** El emisor y el receptor no necesitan estar en ejecución de forma simultánea. Si el servicio de destino se encuentra caído por mantenimiento o saturación, el broker retiene los mensajes de forma segura hasta que el consumidor se restablezca.

2. Nivelación de Carga y Absorción de Picos (*Load Leveling / Peak Shaving*)

En sistemas transaccionales con demandas variables (como una cervecería durante horarios pico de fin de semana), una oleada de peticiones concurrentes puede saturar el pool de conexiones de la base de datos relacional. RabbitMQ actúa como un buffer elástico: las peticiones se encolan inmediatamente a alta velocidad y los consumidores procesan los mensajes a un ritmo constante y controlado (*Competing Consumers Pattern*), protegiendo los recursos críticos aguas abajo.

3. Asincronía y Eliminación de Bloqueos en el Hilo Principal

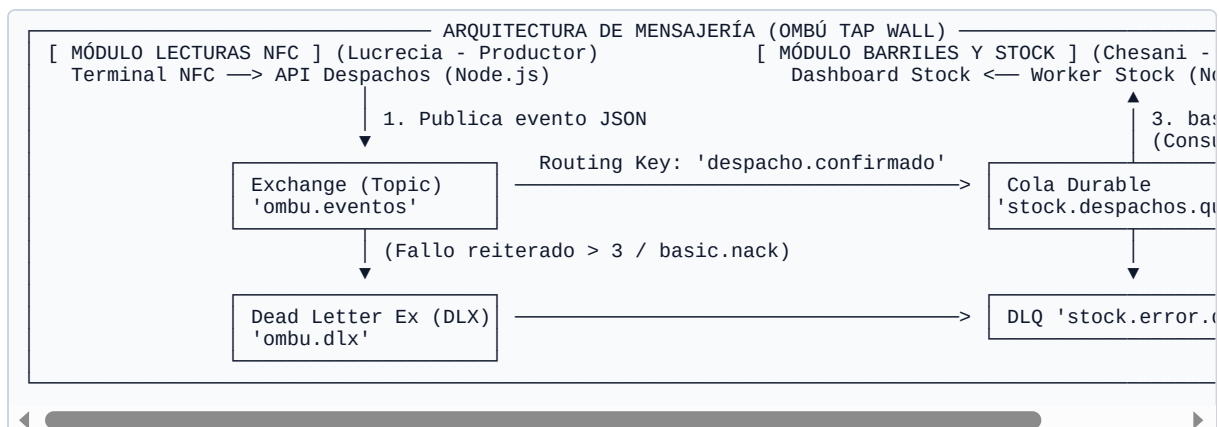
Libera al hilo de ejecución del cliente o API pública de tareas secundarias que no son indispensables para responder de inmediato. El cliente recibe una confirmación inmediata (baja latencia) mientras que los procesos dependientes (cálculo de stock, auditoría, estadísticas) se resuelven asíncronamente en segundo plano.

4. Resiliencia, Tolerancia a Fallos y Garantías de Entrega

A diferencia de llamadas HTTP que fallan si el receptor no responde en pocos segundos (generando excepciones en cadena o *cascading failures*), RabbitMQ garantiza la persistencia mediante escritura en disco, acuses de recibo manuales (*acknowledgments*) y reencolamiento controlado ante fallas de infraestructura.

C. Aplicación Concreta al Proyecto del Grupo (Ombú - Tap Wall)

Trasladamos la arquitectura de mensajería a nuestro sistema de gestión para **Ombú (Cervecería de Barrio)**, enfocado en el muro de canillas autoservicio NFC.



1. Identificación del Evento de Negocio

- **Nombre del Evento:** `DespachoRealizado`
- **Definición de Negocio:** Ocurre en el instante exacto en que un cliente aproxima su tarjeta NFC al lector de una canilla física, el sistema valida que cuenta con saldo disponible, se abre la electroválvula dispensadora y se confirma el volumen de cerveza servido (ej. Pinta de 500 ml).
- **Propiedad y Desacoplamiento:** El módulo de ventas y autoservicio (a cargo de Lucrecia) debita el saldo de la tarjeta y debe responderle al cliente en milisegundos con la apertura del grifo y el ticket. **No debe esperar de forma síncrona que se descuenten los litros del barril en la base de datos.** Por ende, emite el evento `DespachoRealizado` a RabbitMQ.

Estructura del Mensaje (Payload JSON):

```

{
  "eventId": "evt-77f9c2d1-4e8b-4a56-91e2-b88301fa3201",
  "eventType": "DespachoRealizado",
  "occurredAt": "2026-09-23T20:15:30.120Z",
  "idempotencyKey": "idem-nfc-tap-884920-c3",
  "producer": "modulo-nfc-despachos",
  "data": {
    "idDespacho": 105,
    "idCanilla": 3,
    "volumenLitros": 0.500,
    "mililitros": 500,
    "formato": "Pinta",
    "importeDebitado": 2500.00,
    "idTarjeta": 14,
    "idCliente": 2
  }
}
  
```

2. Definición de Componentes en la Arquitectura de Ombú

Componente AMQP	Nombre en Ombú	Tipo / Atributos	Responsabilidad en el Dominio
Productor	Módulo NFC / Despachos (Lucrecia)	Servicio Node.js	Autoriza el débito del cliente y publica el evento sin bloquear la atención del grifo.
Exchange	<code>ombu.eventos</code>	<code>topic</code> (Durable)	Distribuidor central de eventos del Tap Wall.
Routing Key	<code>despacho.confirmado</code>	String contextual	Enruta mensajes de consumo finalizado hacia las colas suscritas.
Cola	<code>stock.despachos.queue</code>	<code> durable: true </code>	Buffer persistente que aloja los consumos pendientes de reflejarse en el stock.
Consumidor	Módulo Stock / Barriles (Chesani)	Worker en segundo plano	Lee los mensajes de la cola, descuenta los litros del barril conectado a la canilla #3 y actualiza el estado a <code>Sin Stock</code> si el barril se vacía.

3. Tratamiento del Fallo e Idempotencia

En sistemas de misión crítica como el control de inventario y facturación de Ombú, es inaceptable que una falla de red desconecte el consumo o que un reintento descuente dos veces la cerveza de un barril:

1. Acuses de Recibo Manuales (*Manual Acknowledgment - basic.ack*):

El worker de stock no utiliza confirmación automática (`noAck: false`). Primero realiza la actualización transaccional en la tabla `Barril` de SQL Server. Solo cuando la sentencia `UPDATE` se confirma con éxito emite `channel.ack(msg)`. Si el proceso se apaga antes de confirmar, el mensaje no se pierde y RabbitMQ lo entrega a otro worker disponible.

2. Control de Idempotencia en el Consumidor:

Dado que RabbitMQ garantiza entrega *al menos una vez* (*at-least-once delivery*), un fallo de red durante el envío del `ack` provocaría que el mensaje se reenvíe. Para evitar descontar litros dos veces por el mismo despacho, el consumidor implementa un control de unicidad:

```
-- Verificación de idempotencia atómica en SQL Server
IF NOT EXISTS (SELECT 1 FROM DespachoProcesadoStock WHERE id_despacho = @idDespacho)
BEGIN
    BEGIN TRANSACTION;
    -- Descontar litros del barril conectado a la canilla indicada
    UPDATE Barril
    SET litros_restantes = litros_restantes - @volumenLitros
    WHERE id_canilla = @idCanilla AND estado = 'Conectado';

    -- Registrar auditoría del despacho ya aplicado
    INSERT INTO DespachoProcesadoStock (id_despacho, fecha_procesado)
    VALUES (@idDespacho, GETDATE());
    COMMIT TRANSACTION;
END
```

3. Dead Letter Queue (DLQ) ante Fallas Irrecuperables:

La cola `stock.despachos.queue` se declara con el argumento `x-dead-letter-exchange: ombu.dlx`. Si un mensaje presenta datos corruptos o falla 3 veces consecutivas, el worker ejecuta `channel.nack(msg, false, false)`, derivándolo automáticamente a la cola `stock.error.dlx` para ser inspeccionado por soporte técnico sin trabar la operación de los clientes en el bar.

Bibliografía

- **Hohpe, G., & Woolf, B. (2004).** *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley. (Capítulo 2: Messaging Channels; Capítulo 3: Message Routing).
- **Fowler, M. (2002).** *Patterns of Enterprise Application Architecture*. Addison-Wesley. (Patrones de concurrencia e integración desacoplada).
- **RabbitMQ Documentation (2026).** *RabbitMQ Tutorials: "Hello World" & "Work Queues" (AMQP 0-9-1 Model)*. <https://www.rabbitmq.com/tutorials>
- **Cátedra Paradigmas y Lenguajes de Programación III (2026).** *Recurso Audiovisual Oficial de Cátedra: Video Tutorial RabbitMQ (Arquitectura y Primeros Pasos)*. UCP - FAITA.